

When Does a Self-Applicable Canonical Normaliser Exist?

A review for a dependently typed theory T and semantics S

16 August 2026

Abstract

Given a type system T and a semantics S , does there always exist a self-applicable normaliser, or even a canonical one? The answer is no without substantial hypotheses. This note separates four questions that are often conflated: existence of normal forms, computable normalization, semantic soundness, and typed self-application. It gives counterexamples showing why arbitrary T and S do not suffice, and states sufficient conditions under which a canonical normaliser exists. A staged code type supplies self-application without requiring an unrestricted self-typing universe.

Contents

1	Question and terminology	1
2	The unrestricted claim is false	2
3	Sufficient conditions for normalization	2
4	Adding semantics S	3
5	Conditions for self-application	3
6	What can be concluded for the implemented core	4
7	Conclusion	4

1 Question and terminology

Let T contain terms t, u , a typing judgment $\Gamma \vdash_T t : A$, and a reduction relation $\rightarrow_T$. Let S assign meaning to well-typed terms, written

$$\llbracket t \rrbracket_S \in \mathcal{M}_A.$$

The phrase “a normaliser exists” has several meanings.

Definition 1.1 (Normalizer). *A normalizer is a partial algorithm N such that, whenever $\Gamma \vdash_T t : A$, $N(\Gamma, t)$ terminates with a normal form n and*

$$\Gamma \vdash_T n : A, \quad t \equiv_T n.$$

Definition 1.2 (Semantic normalizer). *A normalizer is semantically sound for S when*

$$\llbracket N(\Gamma, t) \rrbracket_S = \llbracket t \rrbracket_S.$$

This is weaker than definitional soundness unless S is faithful for the chosen equality.

Definition 1.3 (Canonical normalizer). *A normalizer is canonical when equal well-typed terms produce syntactically equal output, after fixing the representation of binders, universes, quotation, and eta-expansion:*

$$t \equiv_T u \implies N(t) = N(u).$$

Definition 1.4 (Staged self-application). *Suppose T has a typed code family $\text{Code}(A)$ and operations*

$$\text{quote} : A \rightarrow \text{Code}(A), \quad \text{unquote} : \text{Code}(A) \rightarrow A.$$

A normalizer is staged-self-applicable when its implementation has a code value $\hat{N}$ and

$$\text{unquote}(\hat{N}) \text{quote}(t)$$

is well typed and extensionally equal to $N(t)$. The code value is inert until unquoted.

2 The unrestricted claim is false

Proposition 2.1. *For arbitrary type systems T and semantics S , a normalizer need not exist. Consequently, neither a canonical nor a self-applicable normalizer is guaranteed to exist.*

Proof. Several independent counterexamples suffice.

First, let T contain a well-typed general recursive term Ω with $\Omega \rightarrow_T \Omega$. If normalization is required to terminate on every well-typed term and preserve definitional equality, no such normalizer can return a finite normal form for Ω .

Second, let T be terminating but non-confluent, with a term t reducing to two distinct normal forms n_1 and n_2 . A reduction-based normalizer may choose either, but without an additional canonicalization rule there is no reason for conversion-equivalent terms to yield the same syntax. If both normal forms are observationally distinct under S , no single canonical representative can be justified by semantics alone.

Third, let S be an arbitrary, non-computable interpretation. Semantic equality may be undecidable even when the syntax of T is simple. A semantic normalizer computed from S then need not exist as an effective program.

Finally, if T has no code type or no typed execution operation, a normalizer may exist while typed self-application is not even a meaningful judgment. Normalization and self-application are therefore separate properties. $\square$

An inconsistent system can make the question vacuous: if every proposition is derivable, a term may be assigned many types, but that does not provide a terminating, sound computational normalizer. In particular, an unrestricted $\text{Type} : \text{Type}$ is not a safe substitute for staged self-application.

3 Sufficient conditions for normalization

The following theorem gives a standard conditional answer.

Theorem 3.1 (Effective canonical normalization). *Assume the following conditions for T .*

1. **Effective syntax and typing:** *terms, contexts, and typing derivations have finite representations; well-typedness is decidable.*
2. **Strong normalization:** *every well-typed reduction sequence is finite.*
3. **Confluence:** *if $t \rightarrow_T^* u$ and $t \rightarrow_T^* v$, then there is w with $u \rightarrow_T^* w$ and $v \rightarrow_T^* w$.*

4. **Effective normal forms:** normal forms and their alpha-equivalence have decidable representations.
5. **Effective reduction or NbE:** a terminating algorithm computes a normal form for each accepted term.
6. **Stable extensional conventions:** if eta is part of definitional equality, eta-long reification is used; if quotation is present, quoted syntax has a fixed canonical representation.

Then T has a computable canonical normalizer on well-typed terms.

Proof. Strong normalization gives existence of normal forms. Confluence and the Church–Rosser property imply uniqueness of normal forms up to alpha-equivalence. Effective reduction or normalization by evaluation computes one such form. Canonical de Bruijn syntax removes alpha-equivalence, while the fixed eta and quotation conventions remove the remaining representational choices. Therefore equal well-typed terms produce identical output syntax. $\square$

Corollary 3.2 (Soundness). *If the algorithm additionally preserves types across reduction and reification, then*

$$\Gamma \vdash_T t : A \implies \Gamma \vdash_T N(t) : A \quad \wedge \quad t \equiv_T N(t).$$

4 Adding semantics S

The previous theorem is syntactic. To connect it to S , assume a soundness and adequacy pair:

$$t \rightarrow_T u \implies \llbracket t \rrbracket_S = \llbracket u \rrbracket_S,$$

and

$$\llbracket t \rrbracket_S = \llbracket u \rrbracket_S \implies t \equiv_T u$$

for the class of terms under consideration. The first implication says that reduction preserves meaning. The second is faithfulness; it is not automatic and often fails for an abstraction-erasing semantics.

Theorem 4.1 (Semantic correctness). *Under the hypotheses of the effective canonical normalization theorem and semantic soundness,*

$$\llbracket N(t) \rrbracket_S = \llbracket t \rrbracket_S.$$

If S is faithful, semantic equality of two well-typed terms is equivalent to equality of their canonical normal forms.

Proof. $t \equiv_T N(t)$ is a sequence of type-preserving reductions, congruences, and eta conversions. Soundness of S transports equality through each step. Faithfulness gives the converse for the stated class. $\square$

5 Conditions for self-application

Canonical normalization alone does not imply self-application. Add the following staged conditions:

1. a universe-indexed code family $\text{Code}(A)$;
2. typed quotation and unquotation;
3. a representation of the normalizer’s source code at a universe level that contains that code;

4. a stage discipline preventing a code value from being executed before unquotation;
5. preservation of typing by the normalizer transformation.

Theorem 5.1 (Staged self-applicability). *Let T satisfy the effective canonical normalization theorem and the staged conditions above. If the code $\hat{N}$ represents the implementation of N and the code evaluator is adequate for the host evaluator, then*

$$\text{unquote}(\hat{N}) \text{quote}(t) \equiv_T N(t)$$

for every closed well-typed t for which the code application is defined.

Proof. Quotation produces inert syntax. Unquotation restores the normalizer program at the next stage. Adequacy of the code evaluator gives the same computation as the host implementation, and canonical soundness identifies the result with $N(t)$. $\square$

The theorem is deliberately staged. It does not assert an inhabitant of an unrestricted self-function type $N : N \rightarrow N$, and it does not require $\text{Type} : \text{Type}$. Predicative universes may instead provide $\text{Type}_i : \text{Type}_{i+1}$ while placing the code of the normalizer at a higher level.

6 What can be concluded for the implemented core

The implemented C++23 core satisfies the intended prototype conditions for its supported fragment: finite de Bruijn syntax, decidable bidirectional checking, predicative sort formation, substitution, closure-based NbE, neutral reflection, eta-long reification, typed $\text{Code}(A)$ syntax, and an explicit staged normalizer entry point. Its executable tests establish representative normalization, idempotence, substitution, universe rejection, quotation, and staged self-application cases.

The conclusion is conditional rather than universal:

For arbitrary (T, S) : no normalizer is guaranteed.

For an effective, strongly normalizing, confluent T with staged code: a canonical self-applicable normalizer exists.

The semantics S supplies a meaningful interpretation only after soundness has been proved; it cannot by itself manufacture termination, confluence, or computability.

7 Conclusion

There is no theorem saying that every type system and semantics admits a self-applicable canonical normalizer. The correct result is a conditional one. Strong normalization supplies termination, confluence supplies uniqueness, effective syntax supplies computation, canonical representation removes alpha and eta ambiguity, and a typed quotation boundary supplies safe self-application. These obligations should be proved separately when extending the core with inductive types, equality, recursion, or richer metaprogramming.